

ASSESSMENT AND INTERNAL VERIFICATION FRONT SHEET (Individual Criteria)

(Note: This version is to be used for an assignment brief issued to students via Classter)

Course Title	B.Sc. (Hons.) Software Development			Lecturer Name & Surname	Ryan Attard David Deguara	
Unit Number & Title		ITSFT-606-1620-Programming for the Cloud				
Assignment Number, Title / Type		Building a cloud based website				
Date Set			Deadline Date	10th May		
Student Name			ID Number		Class / Group	

Assessment Criteria		Maximum Mark
AA2.1	Choose and apply appropriate data storage solution(s) for a given scenario	7
Ku3.1	Construct a scheduled job in a given scenario	5
KU4.1	Construct a service while making use of a cloud service feature that can be consumed by a 3 rd party	5
AA4.2	Analyse a number of requirements with respect to your application needs (such as caching, website availability, etc) and configure these in your context	7
KU4.3	Arrange a way how to easily upload your artifacts on the cloud	5
AA4.4	Apply a number of security configurations to secure your artifacts	7
SE2.3	Establish and Design a way how your application can handle large data delivery	10
SE4.6	Design and develop a web application that makes use of a number of cloud services while also consuming an implemented API service residing in a different location	10
Total Mark		56

Notes to Students:

- This assignment brief has been approved and released by the Internal Verifier through Classter.
- Assessment marks and feedback by the lecturer will be available online via Classter (<http://mcast.classter.com>) following release by the Internal Verifier
- Students submitting their assignment on Moodle/Turnitin will be requested to confirm online the following statements:

Student's declaration prior to handing-in of assignment

- ❖ I certify that the work submitted for this assignment is my own and that I have read and understood the respective Plagiarism Policy

Student's declaration on assessment special arrangements

- ❖ I certify that adequate support was given to me during the assignment through the Institute and/or the Inclusive Education Unit.
- ❖ I declare that I refused the special support offered by the Institute.

Assignment Guidelines

- This is a home-based assignment.
- Plagiarism and copying are strictly prohibited and will be penalised in accordance with disciplinary regulations.
- Students must use the provided cloud account responsibly.
- Submission deadline: 10 May 2026.
- Important: Marks are allocated for each specified technology. While alternative methods may be used to achieve similar outcomes, failure to use Google cloud or tutor-approved similar cloud technologies will result in a loss of marks for the relevant criteria.
- Marks will be awarded only if the submitted functionality is fully functional and demonstrated. Non-working applications/features will not be graded.
- All submissions must be made through the VLE platform:
<https://vle.mcast.edu.mt/mod/assign/view.php?id=75439>

Scenario

Many restaurants still provide their menus in the form of printed posters, scanned images, or PDF files.

This makes it difficult for customers and hospitality platforms to search for specific dishes or compare prices across different restaurants.

A hospitality technology company wants to develop a **cloud-based system** that allows users to upload restaurant menu images and automatically convert them into searchable digital data using Artificial Intelligence.

The system must make use of **cloud storage, cloud AI services, NoSQL databases, and cloud-based automation**.

User Story

Scenario 1: Uploading a Menu (User – “Emma”)

Emma visits a restaurant and takes a photo of the menu.

She:

- Logs in to the system using her Google account (OAuth)
 - Uploads one or more menu images
 - Sees an upload progress indicator
 - Receives confirmation once processing is complete
 - Can later search for dishes and prices
-

Scenario 2: Searching for Dishes (User – “David”)

David wants to find a specific dish.

He:

- Logs into the system
 - Searches for “Pasta”, “Burger”, or “Steak”
 - Sorts results by price
 - Views which restaurant offers the cheapest option
-

Scenario 3: Menu Updates (System Automation)

Different users may upload menus from the same restaurant at different times.

If prices or dishes change, the system must:

- Detect differences
 - Update the stored data
 - Keep a record of previous values
-

Important Note

All features must be implemented using the **cloud services introduced in the course**. Solutions that bypass cloud services are not considered acceptable. Providing the code only is not acceptable – application must run and provide a smooth user experience.

AA2.1 — Choose and Apply Appropriate Data Storage Solutions

a) Cloud Storage Configuration for Menu Images [2 Marks]

You must configure and use a Cloud Storage bucket with the following properties:

- Storage class: Standard
- Access control: Uniform bucket-level access enabled
- Public access: Disabled
- Location: EU-based region

All uploaded menu images must be stored in this bucket. Evidence must be provided through configuration screenshots and application integration.

a) A NoSQL database must be implemented using a hierarchical structure [3 Marks]:

restaurants → menus → images

Where:

- restaurants store restaurant details
- menus store menu items and latest prices & ocrText
- images store references to uploaded menu images

Marks are awarded for correct nesting and logical separation of data. [2 Marks]

KU4.3 - Arrange a way how to easily upload your artifacts on the cloud

b) Implement a web-based upload feature that allows users to: [2 marks]

- Select and upload multiple menu images in a single operation
- Upload files directly to the configured Cloud Storage bucket

c) A real-time upload progress indicator must be implemented using client-side scripting (e.g. JavaScript). [3 marks]

- Display the progress of each file upload
- Be synchronized with the actual upload process
- Provide clear feedback on completion or failure

SE4.6 - Design and develop a web application that makes use of a number of cloud services while also consuming an implemented API service residing in a different location

- a) When the user uploads the menu images, they should be stored in a bucket (assessed in aa2.1), reference added to a NoSQL database (assessed in aa2.1) and an entry should be added to a PubSub Topic *menu-uploads-topic*. [3 marks]
- b) After added to that topic, a Pub/Sub Cloud Function should be triggered and calls the Cloud Vision API to process the menu image and stores the output back into the NoSQL database under *restaurant/{restaurantId}/menus/{menuId}*, [ocrText] field and restaurant status as "pending" [4]
- c) See KU3.1;

- d) In the catalog/index page show “completed” menu items with a button/dropdown next to each menu item to enable users to translate the menu item in a selected language. This button task (from a user perspective) should connect to the HttpFunction in KU4.1 (via the controller), which returns the translated text and viewed on screen [3 marks].
Note: on a more detailed implementation of this, see AA4.2.

 KU4.1 - Construct a service while making use of a cloud service feature that can be consumed by a 3rd party

- a) Construct an HttpFunction, which uses Translation API. This should work simply accept some text and a selected language, and it translates the content into the selected language returning the translated text. [5 marks]

 AA4.2 - Analyse a number of requirements with respect to your application needs (such as caching, website availability, etc) and configure these in your context

- a) SE4.6 (d), states that a button should trigger the language translation of the menu item. When this button is clicked instead of calling the language Translation API directly first it must check the cache server whether the translation to the selected language for the given menu item exists or not;
- If it exists, we load the translation from the cache [2 marks];
 - If it doesn't exist process must connect to the HttpFunction and the results stored in a structured manner that can be retrieved later in and from the cache [3 marks].
- b) Note: When a menu is uploaded and an older version exists, cache for that menu/restaurant should be invalidated (deleted). [2 marks]

 AA4.4 - Apply a number of security configurations to secure your artifacts (e.g. APIs, Data,etc)

- a) The user must login/ register using OAuth 2.0 for the login; [2 marks]
- Identity classes which facilitate relational database access are not to be used since in SE4.6 a NoSql database has been opted for
 - Secret key for OAuth login should be loaded from Secret Manager [3 marks]
 - Authorization must be in place for someone to upload any images. [2 marks]

 KU3.1 - Construct a scheduled job in a given scenario

- a) Construct a cron job that triggers the Http Function every hour;
- b) The cron job has the task to go through the menus that are still marked “Pending” and cleans the data which means store the menu items and the price in a “structured” way to be searchable and filterable [5 marks]. Once this process is finished, mark the restaurant with status “confirmed” or “ready”;

** SE2.3 - Establish and Design a way how your application can handle large data delivery**

- c) Implement a web user-interface to demonstrate
- File Upload (assessed in KU4.3);
 - Catalog allowing the user to search for menu items, sortable (ascending or descending) by the price [5 marks] while allowing for language translation for the “completed” ones only [assessed in KU4.1 & SE4.6]
- d) All the aforementioned tasks (in all the criteria above) are hosted on the cloud including the user-interface mentioned here which should be hosted in a dedicated container run by Cloud Run. [5 marks]